

CHESS WITH AI

A Project Report

Submitted by

B.LASYA AP23110011611

M.SANTOSH AP23110011562

B.CHAITANYA AP23110011561

K.ABHINAV AP23110011568

Under the Supervision of

K. KAVITHA RANI

Assistant Professor

Department of Computer Science and Engineering

SRM University-AP

In partial fulfilment for the requirements of the Hands on with Python(CSE205) Project

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRM UNIVERSITY-AP

NEERUKONDA

MANAGALAGIRI - 522503

ANDHRA PRADESH, INDIA

APRIL-2025

CERTIFICATE

This is to certify that the project work entitled “[**CHESS WITH AI**]” is a Bonafide record of project work carried out by the following students:

- **Ms. B.LASYA** (AP23110011611)
- **Mr. M.SANTOSH**(AP23110011562)
- **Mr. B.CHAITANYA** (AP23110011561)
- **Mr. K.ABHINAV** (AP23110011568)

from the **Department of Computer Science and Engineering, SRM University-AP**. The students conducted this project work under my supervision during the period **January 2025 to April 2024**. It is further certified that, to the best of my knowledge, this project has not previously formed the basis for the award of any degree or any similar title to this or any other candidate.

This is also to certify that the project work represents the **teamwork** of the candidates.

Station: Mangalagiri
Date:

K. KAVITHA RANI
Assistant Professor
Department of Computer Science & Engineering
SRM University-AP
Andhra Pradesh.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who provided their support and guidance throughout the successful completion of our project “Chess Game with AI using Python”.

First and foremost, we would like to thank K. Kavitha Rani, Assistant Professor, Department of Computer Science and Engineering, SRM University-AP, for his valuable mentorship, insightful suggestions, and constant encouragement throughout the duration of this project.

We are also grateful to our department and faculty members for providing the necessary infrastructure and environment that facilitated the development and testing of our application.

We extend our appreciation to the developers of open-source libraries such as python-chess, ipywidgets, and svgwrite, without which the development of this project would not have been possible.

Finally, we would like to thank our peers, friends, and family members for their continuous support and motivation.

B. LASYA AP23110011611

B. CHAITANYA AP23110011561

K. ABHINAV AP23110011568

M. SANTOSH AP23110011562

TABLE OF CONTENTS

S.NO	TITLE	PAGENO
1	ABSTRACT	5
2	INTRODUCTION	6
3	METHODOLOGY	7-11
	1. Overview and objectives	7
	2. Game Representation	7-8
	3. Static Evaluation Function	8-9
	4. Minimax Search	9-10
	5. Move Ordering	10
	6. Integration & UI with Jupyter Widgets	10-11
	7. Performance and UX Consideration.	11
4	EXTENSIONS AND FUTURE WORK	12
5	CONCLUSION	13

Abstract

This project presents the design and implementation of a **Chess Game with Artificial Intelligence** using Python. The system allows a human player to compete against an AI opponent in a fully interactive environment. It uses the python-chess library to manage the board state, validate moves, and handle game rules.

The AI is built using the **Minimax algorithm** with **alpha-beta pruning**, enabling it to evaluate potential move sequences and make optimal decisions based on a static evaluation function. The interface is designed with ipywidgets in a Jupyter Notebook, providing real-time interaction and dynamic board updates through SVG rendering.

The game handles all standard rules of chess, including check, checkmate, stalemate, and illegal move prevention. The project demonstrates the integration of search algorithms, object-oriented programming, and UI elements to create an intelligent, turn-based application. It serves as both an educational tool and a functional game showcasing the capabilities of AI in classical board games.

INTRODUCTION

The game of chess has long been regarded as a test of strategic thinking and problem-solving skills. With the advancement of artificial intelligence, the development of a chess-playing system has become a benchmark for evaluating the capabilities of AI algorithms. This project, "Chess Game with AI using Python", aims to simulate an interactive chess environment where a user can play against an intelligent AI opponent.

The project is developed using Python and utilizes the python-chess library, which provides comprehensive tools to represent the chessboard, validate moves, and handle game state transitions. The AI engine is powered by the Minimax algorithm with alpha-beta pruning, allowing it to evaluate possible move sequences and select the most optimal move based on the current state of the board.

An intuitive user interface is created using ipywidgets within a Jupyter Notebook environment, enabling users to input their moves, view a dynamically updating board, and receive feedback on game progress. The system is designed to handle legal and illegal inputs, game end conditions such as checkmate or stalemate, and provide an enjoyable and educational chess-playing experience.

This project not only demonstrates the implementation of AI in game development but also emphasizes modular programming, object-oriented design, and real-time user interaction using Python.

Methodology

This project implements a turn-based chess game in Jupyter Notebook where the human plays White against a simple AI opponent. The AI uses:

1. **Static board evaluation** (material count + terminal checks).
2. **Depth-limited minimax** search.
3. **Alpha-beta pruning** to eliminate unneeded branches.
4. A lightweight **ipywidgets** front-end for move entry and display.

The modular design cleanly separates core engine logic (state, evaluation, search) from the interactive layer, making future improvements—better heuristics, quiescence search, transposition tables—straightforward to add.

1. Overview & Objectives

- **Goal:** Create a functional chess engine capable of playing legal moves and providing a reasonable challenge to a human, integrated into an interactive notebook environment.
- **Scope:**
 1. Represent the chess game state and moves.
 2. Compute a static evaluation of positions.
 3. Search the game tree using minimax with alpha–beta pruning.
 4. Offer an interactive UI for move entry and display.
 5. Outline strategies for improving strength and speed.

2. Game Representation

2.1 Board State

Use `chess.Board()` to maintain:

- o Piece placement on 64 squares
- o Side to move (white/black)
- o Castling rights, en passant targets, half-move clock, full-move number.

2.2 Move Encoding & Validation

- **UCI Format** (e2e4, g1f3) for both parsing user input and outputting AI choices.
- Validate with `chess.Move.from_uci(move_str)` and membership in `board.legal_moves`.
- Leverage `board.push(move)` and `board.pop()` for hypothetical exploration in search.

Why python-chess?

- Correctly enforces legal moves, checks, and game-over conditions.
- Provides easy SAN/UCI conversion.
- Speeds development by offloading low-level bitboard manipulation.

3. Static Evaluation Function

When the search reaches its depth limit or a terminal state, we assign a numeric score:

1. Terminal States

o Checkmate:

- If it's White's turn and the position is checkmate → Black just delivered mate → score = -9999.
- Vice versa → score = +9999.

o Drawish (stalemate, insufficient material, threefold repetition): score = 0.

2. Material Balance

Each piece is given a fixed integer value (centipawns):

```
piece_values = {
    chess.PAWN: 100,
    chess.KNIGHT: 320,
    chess.BISHOP: 330,
    chess.ROOK: 500,
    chess.QUEEN: 900,
    chess.KING: 20000
}
```

We iterate over all 64 squares, summing up +value for White's pieces and -value for Black's.

Terminal Conditions:

- o Checkmate: large $\pm$ score to denote decisive win/loss

- o Stalemate/Insufficient Material: zero (draw)
- o

Material Sum: Weighted sum of white minus black. Simplistic but effective for testing AI logic.

Why Simple Material?

- Fast to compute.
- Captures the bulk of tactical imbalances.
- Easy to extend later (add piece-square tables, mobility, king safety).

4. Minimax Search

4.1 Basic Minimax

Recursive depth-limited search:

1. If depth = 0 or terminal, return evaluate_board().
2. For each legal move:
 - Push move to board
 - Recurse with depth - 1, toggling maximizing flag
 - Pop move
3. Choose max (if maximizing) or min (if minimizing) evaluation and record the move.

4.2 Pseudocode

```
function minimax(board, depth, α, β, maximizingPlayer):
    if depth == 0 or board.is_game_over():
        return evaluate_board(board), None

    if maximizingPlayer:
        maxEval = -∞
        bestMove = None
        for move in board.legal_moves:
            board.push(move)
            eval, _ = minimax(board, depth-1, α, β, False)
            board.pop()
            if eval > maxEval:
                maxEval, bestMove = eval, move
            α = max(α, eval)
            if β <= α:
                break # β cutoff
        return maxEval, bestMove
    else:
        minEval = +∞
        bestMove = None
        for move in board.legal_moves:
            board.push(move)
            eval, _ = minimax(board, depth-1, α, β, True)
            board.pop()
            if eval < minEval:
                minEval, bestMove = eval, move
            β = min(β, eval)
            if β <= α:
                break # α cutoff
        return minEval, bestMove
```

4.3 Alpha–Beta Pruning:

- Reduces search space by pruning branches that cannot affect the final decision: if $\alpha \geq \beta$, no need to explore further.
- Empirically speeds up search by an order of magnitude compared to naive minimax.

5. Move Ordering (Optional Enhancement)

Better move ordering dramatically increases pruning:

- **Captures first** (especially high-value-victims-from-low-value-attackers).
- **Checks** early, forcing the opponent to respond.
- **Sorting** child nodes by a heuristic (e.g. “most-valuable victim, least-valuable attacker”).

Implementing simple MVV–LVA ordering in the move loop can often double effective search depth.

6.Integration & UI with Jupyter Widgets

6.1 Widgets Setup

- **Text Input** for UCI moves.
- **Button** to submit.
- **Output** widget to display board, feedback.

```
move_input = widgets.Text(...)
submit_button = widgets.Button(...)
output = widgets.Output()
```

6.2 Display Function

- Clear and re-render board each turn.
- Use `chess.svg.board()` (if available) for a clean SVG image, else a text board.
- Show last user move, last AI move, turn, and final result when game over.

6.3 Event Handler

- On button click:

1. Parse and validate user move.
2. Push and display.
3. If not over, run `get_ai_move()` → push and display.
4. Handle illegal or invalid input with clear messages.

7. Performance & UX Considerations

- Depth vs. Speed: Depth 3 is ~4–5 seconds per move in pure Python. Depth 2 or adding a simple timing cutoff can keep interactivity snappy.
- Asynchronous Thinking: Offload the search to a background thread or asyncio coroutine to keep the notebook responsive.
- Progress Bars: Provide move-count or depth-count feedback for deep searches.
- Error Recovery: Catch exceptions in the callback to avoid halting the widget loop.

Extensions & Future Work

1. **Better Heuristics**

- o Piece-square tables for positional nuance.
- o Pawn structure, king safety, mobility scores.

2. **Quiescence Search**

- o Extend leaf nodes by exploring only captures to avoid “horizon effect.”

3. **Transposition Tables**

- o Hash board states with Zobrist hashing to reuse evaluations.

4. **Iterative Deepening**

- o Search depths 1,2,...N in sequence, retaining best move when time runs out.

5. **GUI Migration**

- o Integrate with a web front-end (Flask, React) for broader accessibility.

CONCLUSION

This methodology presents a concise yet powerful framework for building a chess AI in Python. By leveraging the `python-chess` library, it cleanly separates rule management from the core AI logic, allowing you to focus on crafting evaluation and search routines without reimplementing the game's mechanics.

The static evaluation function offers a fast, material-based heuristic that serves as a solid foundation. Encapsulating it in a single function ensures maintainability and makes it trivial to introduce more advanced features—like piece-square tables, mobility metrics, or even neural-network evaluations—without touching the search code.

Our minimax search with alpha–beta pruning exemplifies classic adversarial algorithms: a depth-limited recursive exploration with bound-based cutoffs to prune irrelevant branches. This modular design, combined with Python's straightforward push/pop API, makes it both educational and extensible for hobbyists and students alike.

Wrapping the engine in a Jupyter Notebook via `ipywidgets` transforms it into an interactive learning tool. Users can adjust search depth, observe AI thinking times, and inspect board states in real time. Looking ahead, features like transposition tables, move-ordering heuristics, quiescence search, and opening/endgame databases can slot seamlessly into this architecture, guiding you toward a more robust and versatile chess AI.